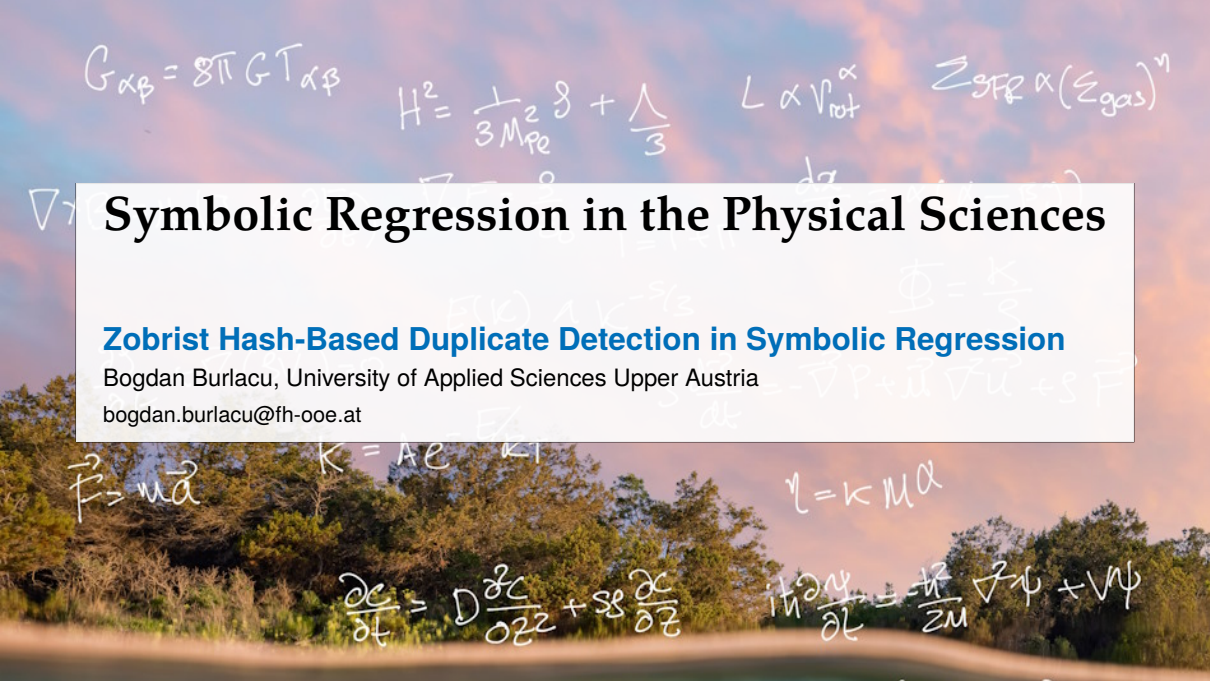


Symbolic Regression in the Physical Sciences

Zobrist Hash-Based Duplicate Detection in Symbolic Regression

Bogdan Burlacu, University of Applied Sciences Upper Austria
bogdan.burlacu@fh-ooe.at

About Me

Professor of Data Science and Machine Learning

University of Applied Sciences Upper Austria

Heuristic and Evolutionary Algorithms Laboratory

Research Interests

- symbolic regression, physical modeling, simulation and optimization
- explainability and interpretable machine learning
- algorithm design, scalability, energy-efficient AI
- chess engine programming
- author of the Operon library
  `https://github.com/heal-research/operon`

Zobrist Hashing

Technique used in abstract board games to implement transposition tables.

- Starts by randomly generating bitstrings for each possible element of a board game (i.e. for each combination of a piece and a position)
- The Zobrist hash is computed by combining those bitstrings using bitwise XOR
- If the bitstrings are long enough, the probability of collisions is negligible

Bitwise XOR → incremental hash update

$$a \oplus b = b \oplus a \qquad \text{(associativity)}$$

$$a \oplus (b \oplus c) = (a \oplus b) \oplus c \qquad \text{(commutativity)}$$

$$a \oplus b \oplus b = a \qquad \text{(involution)}$$

Zobrist Hashing

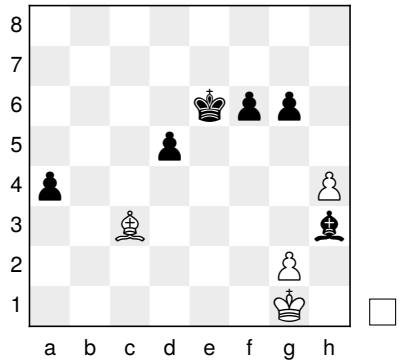
Example

The word “cat” contains three symbols: c, a, t. We associate some random values to each letter:

a	485
c	688
t	118

- $\mathcal{H}(\text{“cat”}) = 688 \oplus 485 \oplus 118 = 803$
- hashing lowercase words up to length m from an alphabet of size n will require a $n \times m$ Zobrist table (filled with random values)
- for a board game: n squares $\times$ m piece types
  - Chess: $n = 64, m = 12$
  - Go: $n = 361, m = 2$
  - Symbolic regression: (number of symbols) $\times$ (max expression size)

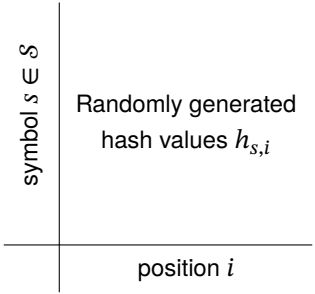
Zobrist Hashing



♙	2173	1288	1235	2880	3217
♟	1074	1570	3800	1538	1260
♘	3658	1022	1433	1801	2504
♞	4054	3302	2029	1632	2549
⋮	⋮	⋮	⋮	⋮	⋮
♔	1909	3847	3117	2965	2533
♚	1270	1694	1986	3800	4014
	0	**1**	**2**	**...**	**63**
	a_1	b_1	c_1	...	h_8

$$\mathcal{H}(\text{position}) = h_{\text{♔}g_1} \oplus h_{\text{♙}g_2} \oplus h_{\text{♗}c_3} \oplus h_{\text{♝}h_3} \oplus h_{\text{♙}h_4} \oplus h_{\text{♟}a_4} \oplus h_{\text{♟}d_5} \oplus h_{\text{♚}e_6} \oplus h_{\text{♟}f_6} \oplus h_{\text{♟}g_6}$$

Zobrist Hashing



$$\mathcal{H}(\text{state}) = \bigoplus h_{s,i}$$

If the state of a program[1] can be described by a list of symbols and their associated positions, then it can be efficiently maintained using Zobrist hashing.

[1] algorithm, search method, simulation, optimization procedure

Fast recall of state-history in kinetic Monte Carlo simulations utilizing the Zobrist key

D.R. Mason [a], T.S. Hudson [a,*], A.P. Sutton [a,b]

[a] *Department of Materials, Oxford University, OX1 3PH, UK*
[b] *Helsinki University of Technology, Laboratory of Computational Engineering, PO Box 9203, FIN 02015 ESPOO, Finland*

Accepted 3 September 2004

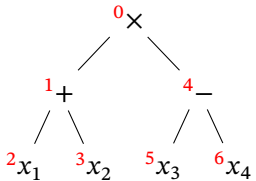
Available online 11 November 2004

Abstract

We present a novel application of the Zobrist hashing method, known in the computer chess literature, to simulation of diffusional phase transformations in metal alloys. A history of previously visited states can be easily maintained, allowing very fast lookup of energies and transition rates calculated earlier in the simulation. The method has been applied to the simulation

Zobrist Hashing in Symbolic Regression

Expression tree T



$$f(\mathbf{x}) = (x_1 + x_2) \times (x_3 - x_4)$$

Prefix representation:

$$\times + x_1\ x_2 - x_3\ x_4$$

0 1 2 3 4 5 6

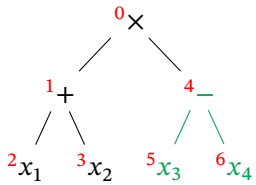
Zobrist table:

$+$	Filled with random values
$-$	
$\times$	
$\div$	
$\vdots$	
x_1	
x_2	
x_3	
x_4	
c	
	position $i = 0, \ldots, N$

$$\mathcal{H}(T) = h_{\times,0} \oplus h_{+,1} \oplus h_{x_1,2} \oplus h_{x_2,3} \oplus h_{-,4} \oplus h_{x_3,5} \oplus h_{x_4,6}$$

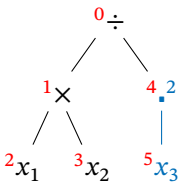
Zobrist Hashing in Symbolic Regression

Parent tree T_1



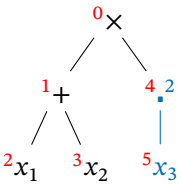
$f_1(\mathbf{x}) = (x_1 + x_2) \times (x_3 - x_4)$

Parent tree T_2



$f_2(\mathbf{x}) = x_1 x_2 / x_3^2$

Child tree C_1



$f_3(\mathbf{x}) = (x_1 + x_2)x_3^2$

Incremental hash update after crossover

$$\mathcal{H}(C_1) = \mathcal{H}(T_1) \oplus \underbrace{h_{-,4} \oplus h_{x_3,5} \oplus h_{x_4,6}}_{\text{XOR "out" } (x_3 - x_4)} \oplus \underbrace{h_{\blacksquare^2,4} \oplus h_{x_3,5}}_{\text{XOR "in" } x_3^2}$$

Zobrist Hashing in Symbolic Regression

The Zobrist hash provides a means to cache duplicate structures that may occur

- We could cache solution candidates and their fitness
- We could use this information to guide the search

Why do duplicates occur in the first place?

- The search process is driven by “natural selection”
- Selection probability proportional with fitness (i.e., training error)
- Fit solution candidates will propagate (“survival of the fittest”)

Zobrist Hashing in Symbolic Regression

Initialization

1. Generate N random trees
2. Evaluate their fitness on the training data

Main loop

1. **Selection**: choose N parent pairs for recombination
2. **Recombination**: produce N children via crossover/mutation
3. **Fitness evaluation**: compute fitness for child individuals
4. **Replacement**: child individuals replace parents
5. **Check stop**: if not converged, go to step 1, otherwise stop.

Zobrist Hashing in Symbolic Regression

Initialization

1. Generate N random trees
2. Evaluate their fitness on the training data

Main loop

1. **Selection**: choose N parent pairs for recombination
2. **Recombination**: produce N children via crossover/mutation $\Longrightarrow$ promote novelty
3. **Fitness evaluation**: compute fitness for child individuals $\Longrightarrow$ cache fitness
4. **Replacement**: child individuals replace parents
5. **Check stop**: if not converged, go to step 1, otherwise stop.

Zobrist Hashing in Symbolic Regression

Novelty promotion

If a child individual's Zobrist signature is already in the cache, discard it and produce a new one

Crossover:

- check all possible subtree exchanges between two parent trees T_1 and T_2
- discard exchanges which would lead to an already-seen hash

Mutation:

- for a parent tree T_1, perform a random change and update hash, discard already-seen hashes

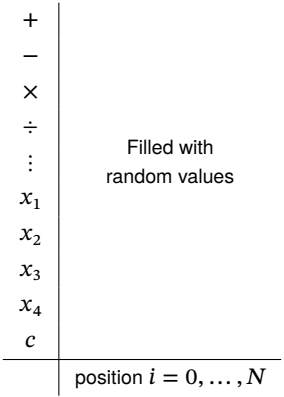
Fitness cache

If child Zobrist hash already in the cache, return stored fitness (do not re-evaluate)

Zobrist Hashing in Symbolic Regression

Implementation (Operon C++)

- Global Zobrist table (singleton class) initialized at program start
- Parallel hash table mapping Zobrist signature to fitness values

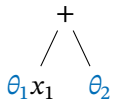


Zobrist Hashing in Symbolic Regression

Implementation (Operon C++)

- Global Zobrist table (singleton class) initialized at program start
- Parallel hash table mapping Zobrist signature to fitness values

Model coefficients not included in hash signature → must be optimized beforehand



Coefficient optimization:

- Nonlinear least squares (Levenberg-Marquardt)
- Configurable number of gradient descent iterations

Test problems

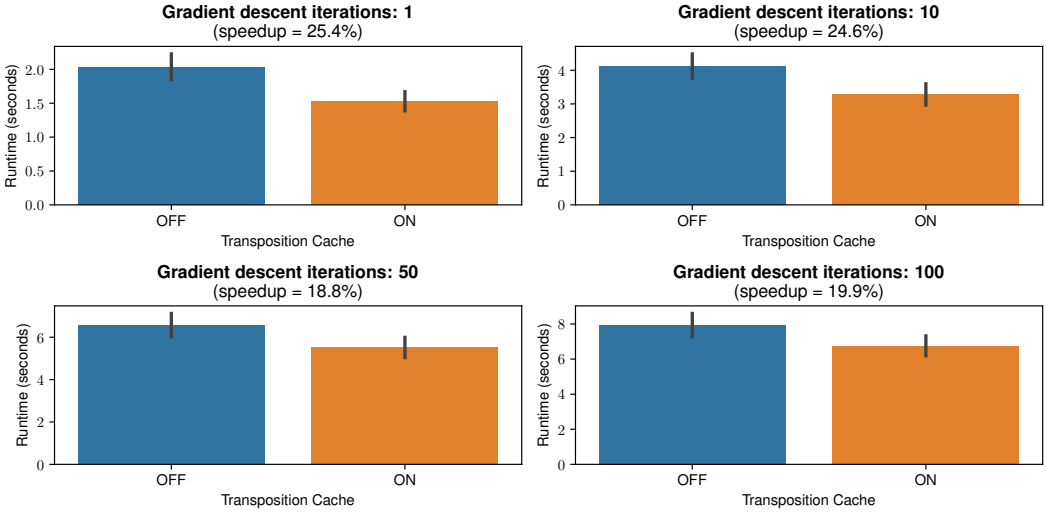
Problem	Points	Features	Train	Test
Chemical-I	4999	25	3136	1863
Chemical-II	1066	57	711	355
FrictionDyn-OneHot	2016	17	1010	1016
FrictionStat-OneHot	2016	16	1010	1016
Flow Stress	7800	2	4200	3600
Nasa Battery 1_10	636	6	504	132
Nasa Battery 2_10	1638	5	1101	537
Nikuradse 1	362	2	230	132
Nikuradse 2	362	1	230	132

Experimental Setup

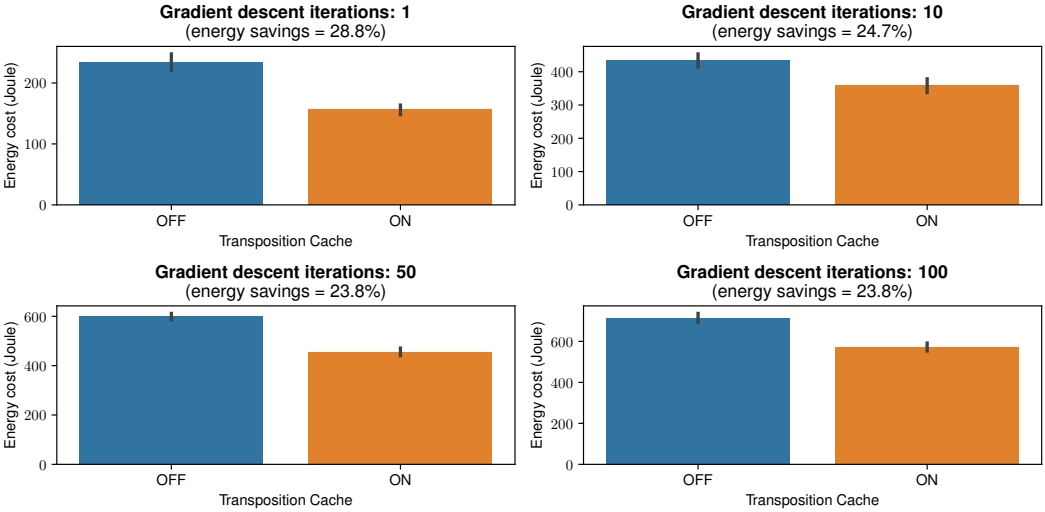
NSGA-II Algorithm

- 50 independent runs for each problem
- 1000 individuals, 300 generations
- varying number of LM-iterations (1, 10, 50, 100)
- transposition cache=on/off, transposition-operators=on/off
- executed on 16 threads (Ryzen 5950X)

Preliminary Results – Runtime



Preliminary Results – Energy Cost



Preliminary Results

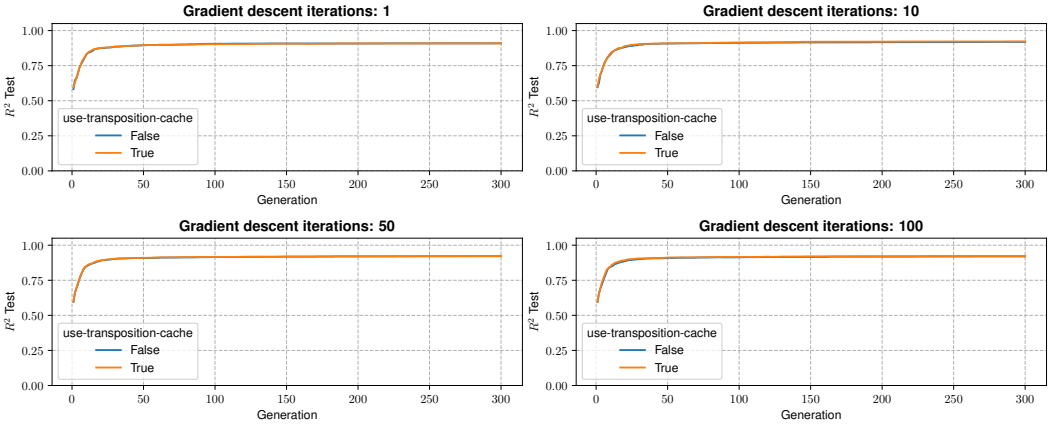


Figure: Chemical-I Test Performance

Preliminary Results

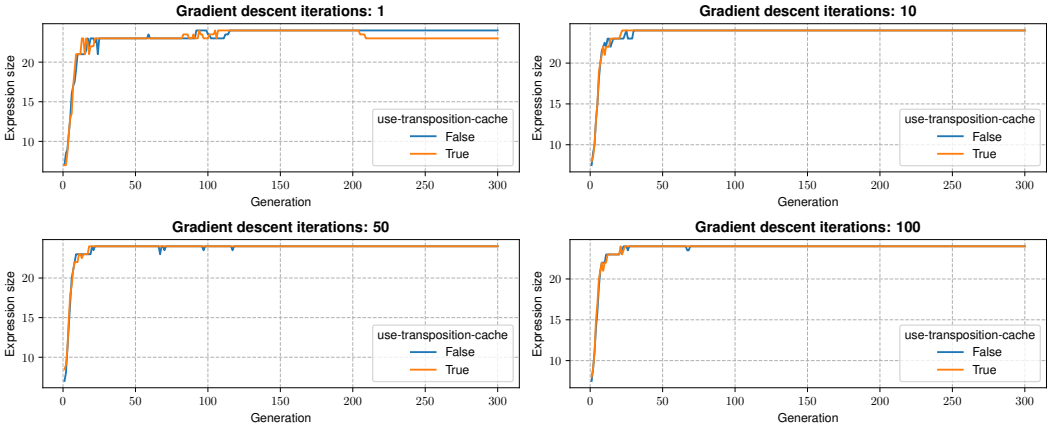


Figure: Chemical-I Expression Size

Preliminary Results

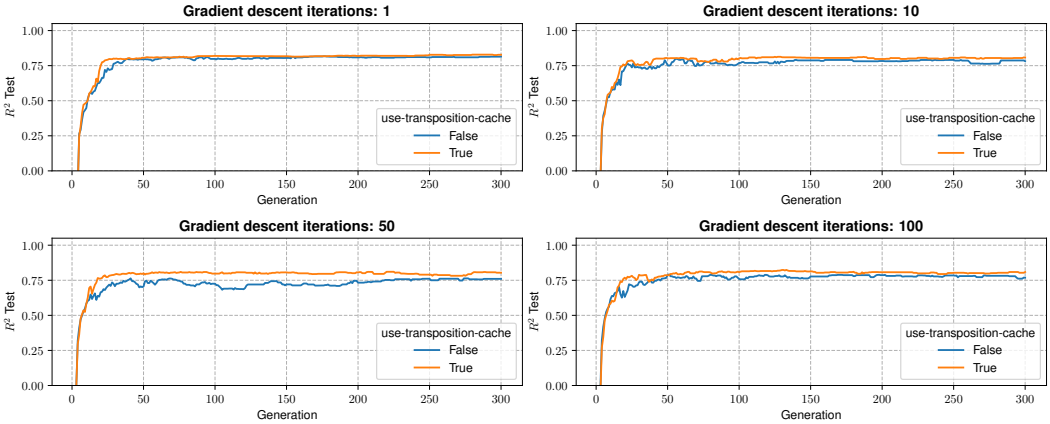


Figure: Chemical-II Test Performance

Preliminary Results

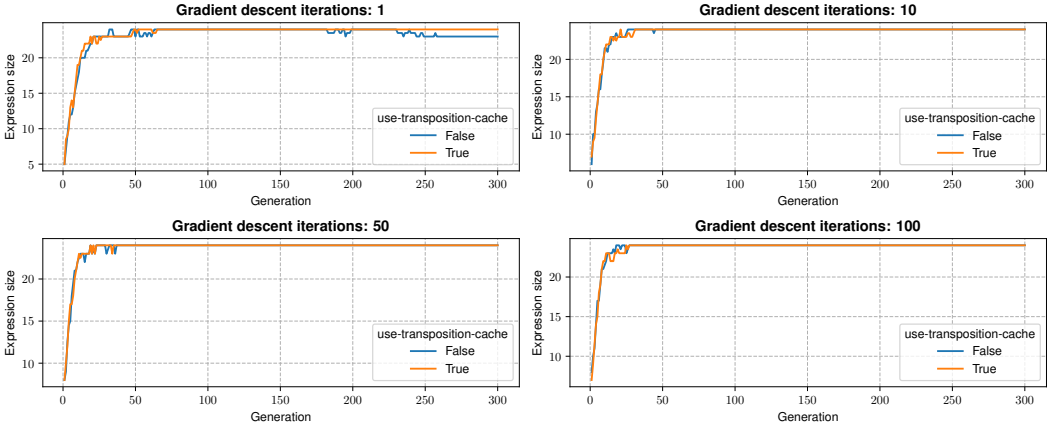


Figure: Chemical-II Expression Size

Preliminary Results

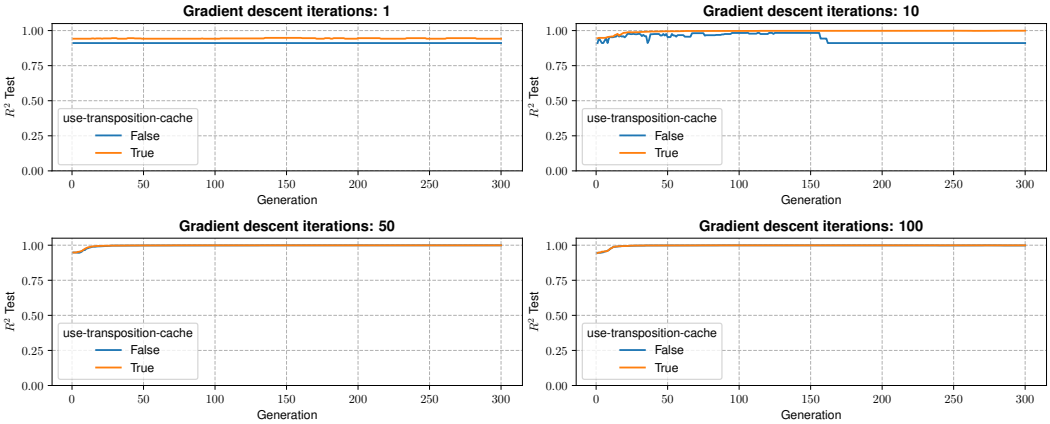


Figure: Flow Stress Test Performance

Preliminary Results

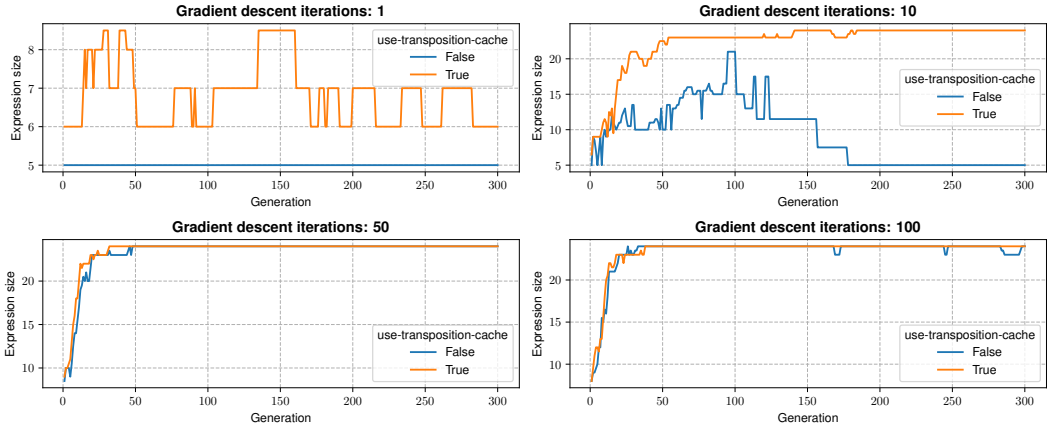


Figure: Flow Stress Expression Size

Preliminary Results

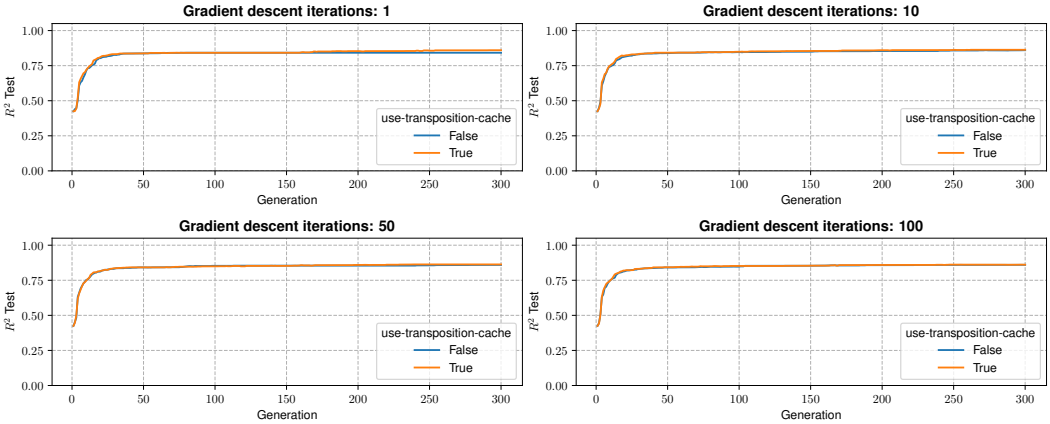


Figure: FrictionDyn-OneHot Test Performance

Preliminary Results

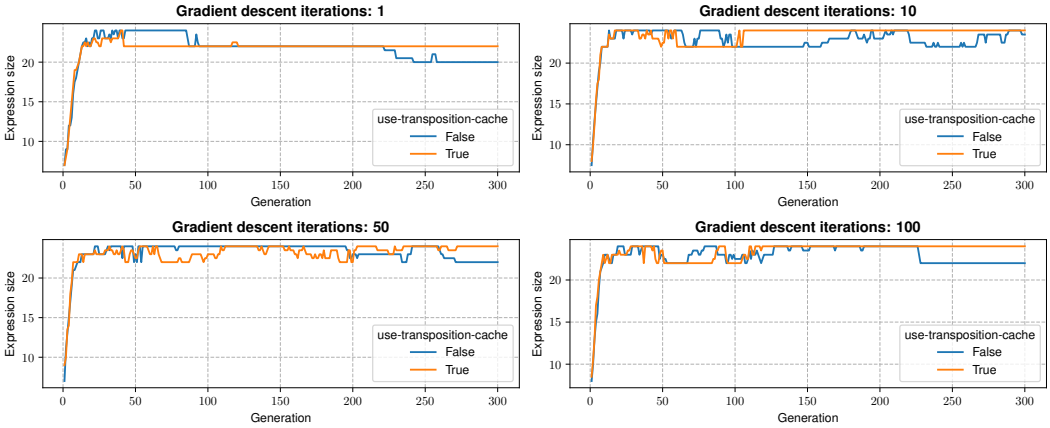


Figure: FrictionDyn-OneHot Expression Size

Preliminary Results

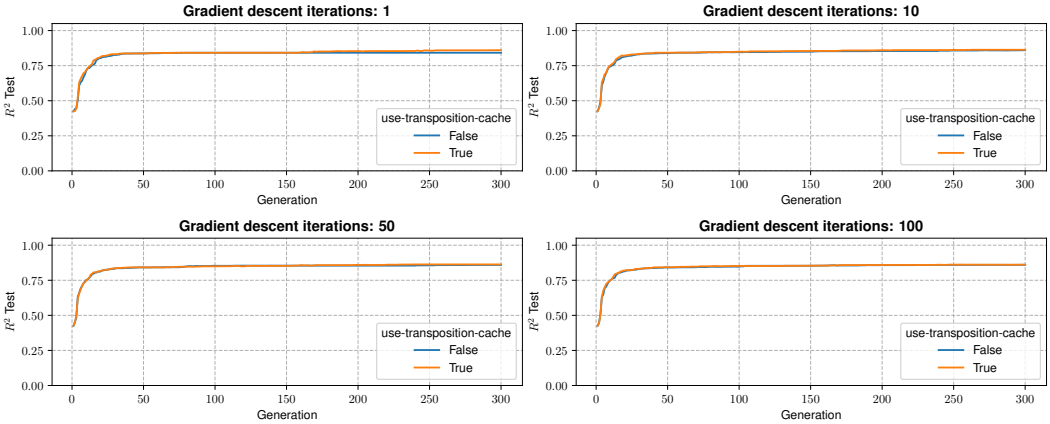


Figure: FrictionStat-OneHot Test Performance

Preliminary Results

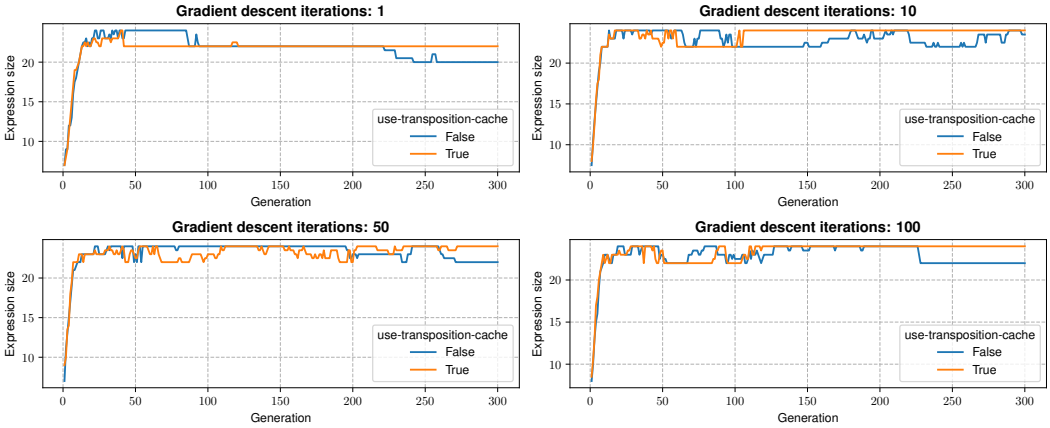


Figure: FrictionStat-OneHot Expression Size

Preliminary Results

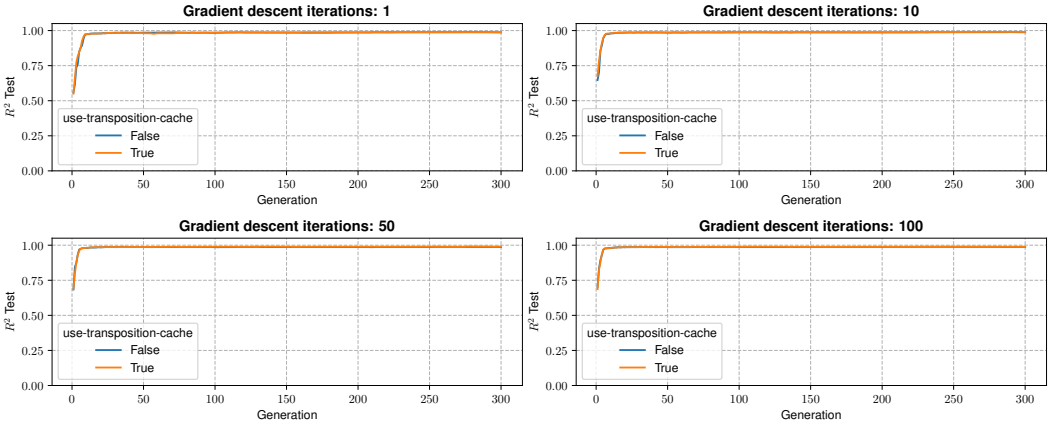


Figure: Nasa Battery 1_10 Test Performance

Preliminary Results

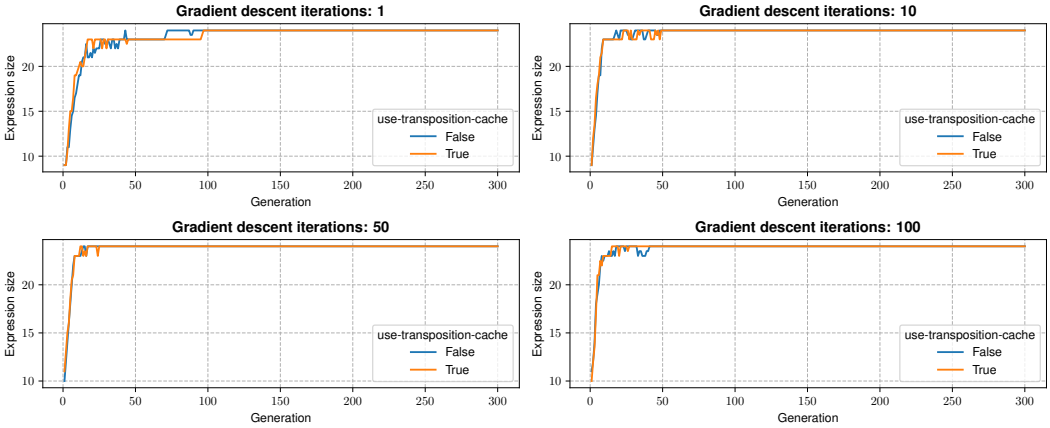


Figure: Nasa Battery 1_10 Expression Size

Preliminary Results

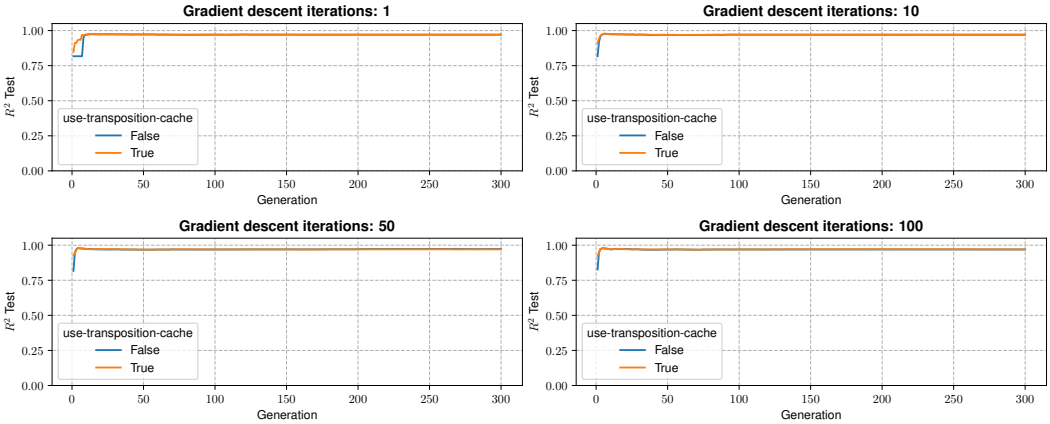


Figure: Nasa Battery 2_20 Test Performance

Preliminary Results

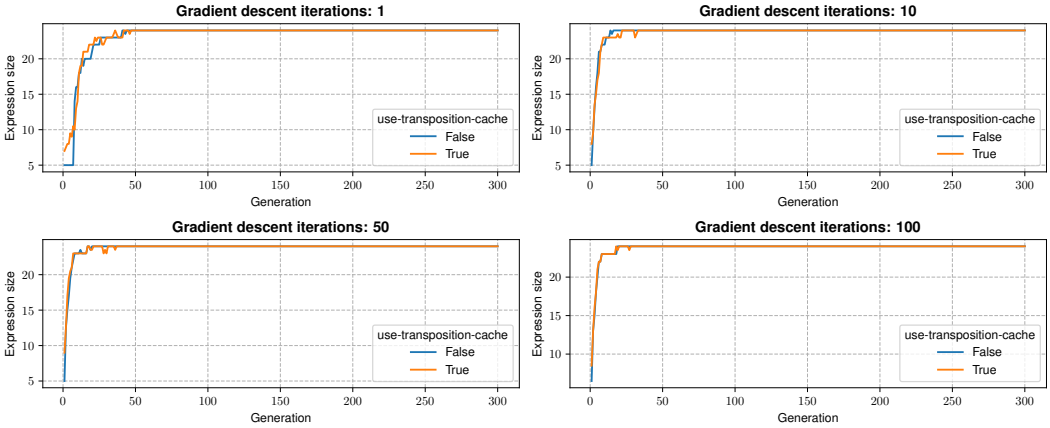


Figure: Nasa Battery 2_20 Expression Size

Preliminary Results

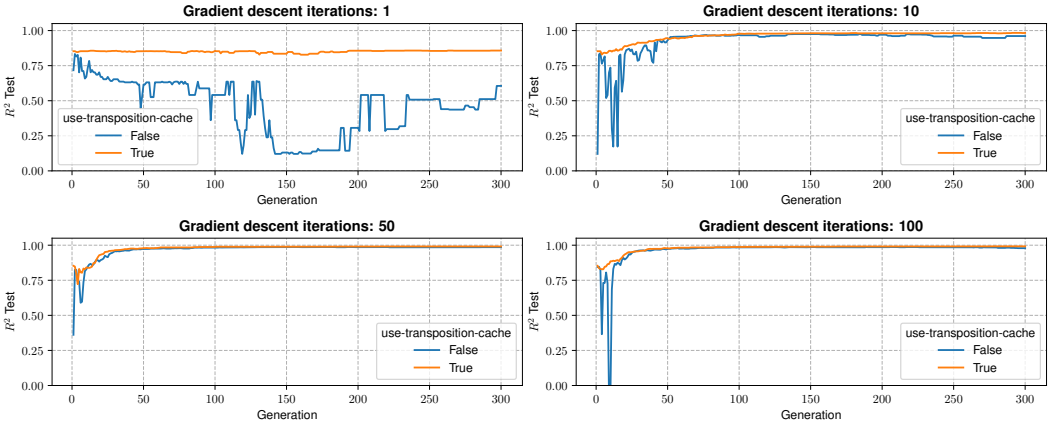


Figure: Nikuradse-I Test Performance

Preliminary Results

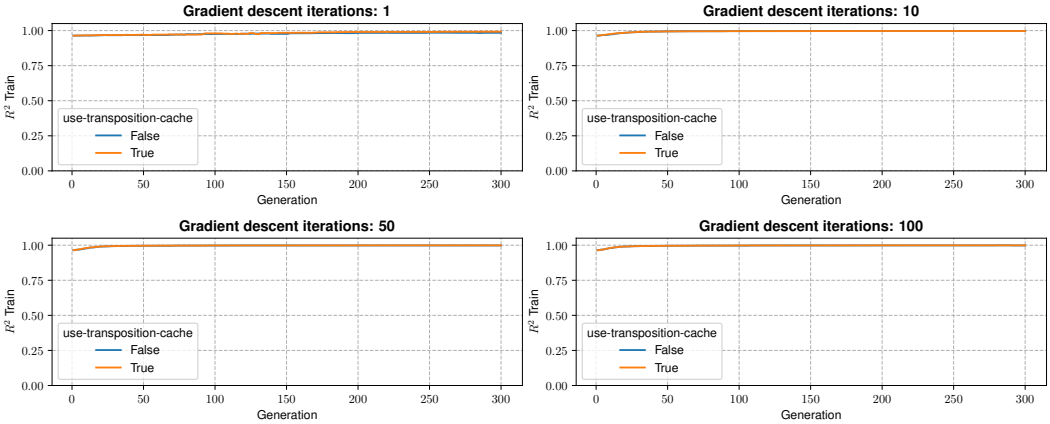


Figure: Nikuradse-I Train Performance

Preliminary Results

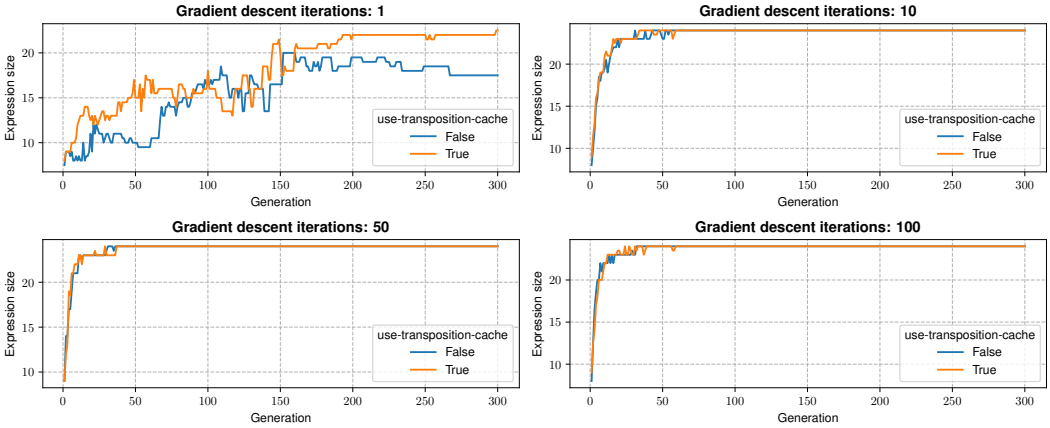


Figure: Nikuradse-I Expression Size

Preliminary Results

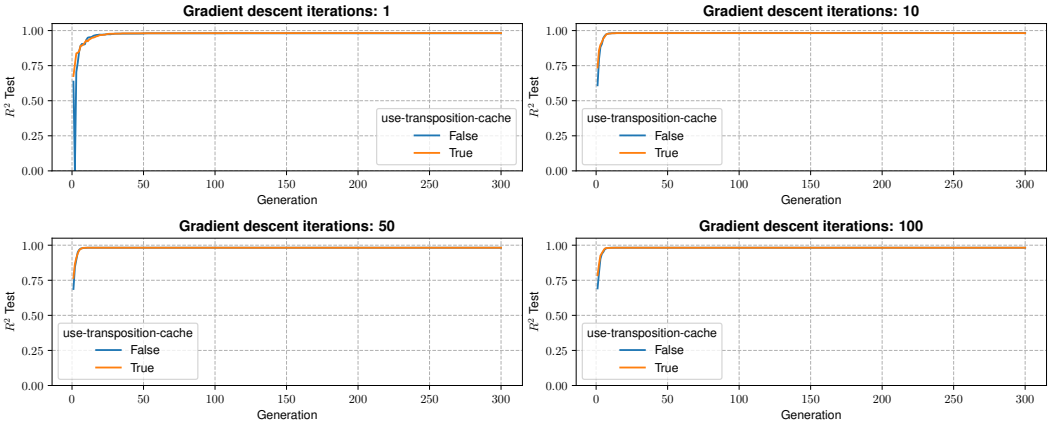


Figure: Nikuradse-II Test Performance

Preliminary Results

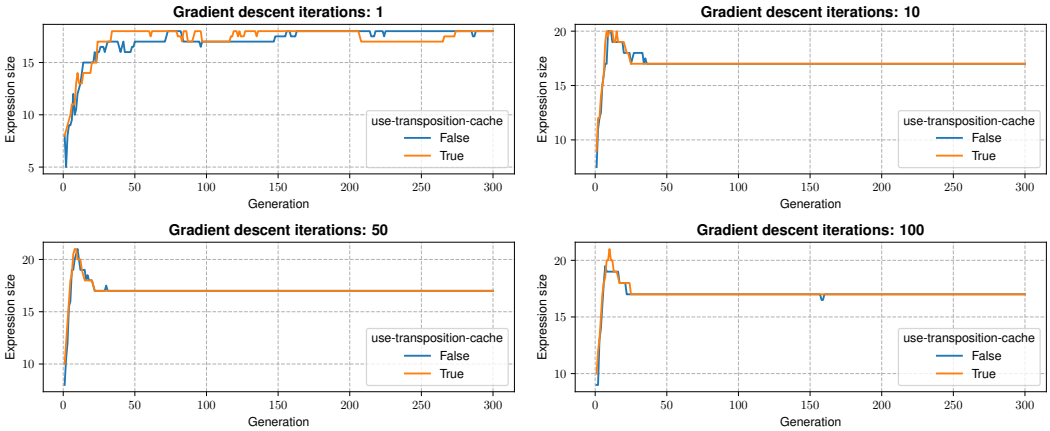


Figure: Nikuradse-II Expression Size

Conclusions

Zobrist Hashing

- Low overhead, easy to implement, resulting in speedups and energy savings
- Fitness caching may induce a more beneficial fitness landscape and prevent overfitting
- Novelty promotion not as reliable in improving performance

Future Work

- Deeper look at algorithm dynamics with caching
- Improved caching strategies: LRU cache, generational
- Better integration with search operators to promote novelty
- Attempt to capture symmetries and other equivalent sub-expressions